Graph Algorithms

> → **Key Idea**: Model problems as graphs and use traversal/search algorithms to find solutions

> • **When to use**: Connectivity, shortest path, cycle detection, topological ordering

> ⊕ **Complexity**: BFS/DFS: O(V+E), Dijkstra: O(E log V), MST: O(E log V)

> ⚠ **Common Pitfall**: Not handling disconnected components or choosing wrong algorithm

Overview

Graph algorithms operate on vertices (nodes) connected by edges. Key categories include traversal (BFS/DFS), shortest path (Dijkstra, Bellman-Ford), minimum spanning tree (Kruskal, Prim), and topological sorting.

Visual Example

BFS vs DFS Traversal Comparison

```
Graph Structure:
      0
     / \
    1   2
   / \   \
  3   4   5

DFS Traversal (Depth-First):   BFS Traversal (Breadth-First):
   Path: 0 -> 1 -> 3           Layers:
         (backtrack)             L0: [0]
         0 -> 1 -> 4             L1: [1, 2]
         (backtrack)             L2: [3, 4, 5]
         0 -> 2 -> 5

   Stack: [0] -> [1,2]         Queue: [0] -> [1,2] -> [3,4,5]
          -> [1] -> [3,4]...
```

Dijkstra's Shortest Path Algorithm

```
Weighted Graph:
    (A)---4---(B)
     |  \      |
```

```
     2    1    3
     |      \  |
    (C)---5---(D)

Finding shortest path from A to all nodes:

Step  | Current | dist[A] | dist[B] | dist[C] | dist[D] | PQ
------|---------|---------|---------|---------|---------|--------
Init  |    -    |    0    |   INF   |   INF   |   INF   | [(A,0)]
  1   |    A    |    0    |    4    |    2    |    1    | [(D,1),(C,2),(B,4)]
  2   |    D    |    0    |    4    |    2    |    1    | [(C,2),(B,4)]
  3   |    C    |    0    |    4    |    2    |    1    | [(B,4)]
  4   |    B    |    0    |    4    |    2    |    1    | []

Shortest paths: A→A=0, A→B=4, A→C=2, A→D=1
```

Topological Sort (DAG)

```
Dependency Graph (Course Prerequisites):
    CS101 → CS201 → CS301
              ↓       ↓
           CS202 → CS302

In-degree calculation:
  CS101: 0  CS201: 1  CS301: 1
  CS202: 1  CS302: 2

Topological Order: CS101 → CS201 → CS202 → CS301 → CS302
                   (or)   CS101 → CS201 → CS301 → CS202 → CS302

Process: Remove node with in-degree 0, update neighbors
```

Cycle Detection in Directed Graph

```
Graph with cycle:
    A → B → C
    ↑       ↓
    └── E ← D

DFS States: WHITE (unvisited), GRAY (in progress), BLACK (done)

Visit A (GRAY) → B (GRAY) → C (GRAY) → D (GRAY) → E (GRAY)
E points to A which is GRAY → CYCLE DETECTED!

Rule: Back edge to GRAY node = cycle exists
```

Minimum Spanning Tree (Kruskal's)

```
Graph:                          MST Result:
    A---4---B                       A       B
    |\      |                       |       |
    2  1    3                       2       3
    |    \  |                       |       |
    C---5---D                       C---1---D

Edges sorted: (A,D)=1, (A,C)=2, (B,D)=3, (A,B)=4, (C,D)=5

Process:
1. Add (A,D)=1 ✓ (no cycle)
2. Add (A,C)=2 ✓ (no cycle)
3. Add (B,D)=3 ✓ (no cycle)
4. Skip (A,B)=4 (would create cycle A-B-D-A)
5. Skip (C,D)=5 (would create cycle)

MST Total Weight: 1 + 2 + 3 = 6
```

Core Implementation

BFS Template

```java
public void bfs(int start, List<List<Integer>> graph) {
    boolean[] visited = new boolean[graph.size()];
    Queue<Integer> queue = new LinkedList<>();

    queue.offer(start);
    visited[start] = true;

    while (!queue.isEmpty()) {
        int node = queue.poll();
        // Process node

        for (int neighbor : graph.get(node)) {
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                queue.offer(neighbor);
            }
        }
    }
}
```

DFS Template

```java
public void dfs(int node, List<List<Integer>> graph, boolean[] visited) {
    visited[node] = true;
    // Process node
```

```
    for (int neighbor : graph.get(node)) {
        if (!visited[neighbor]) {
            dfs(neighbor, graph, visited);
        }
    }
}
```

Dijkstra's Algorithm

```
public int[] dijkstra(int start, List<List<int[]>> graph) {
    int n = graph.size();
    int[] dist = new int[n];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[start] = 0;

    // Min-heap: (distance, node)
    PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
    pq.offer(new int[]{0, start});

    while (!pq.isEmpty()) {
        int[] curr = pq.poll();
        int d = curr[0], u = curr[1];

        if (d > dist[u]) continue; // Skip outdated entries

        for (int[] edge : graph.get(u)) {
            int v = edge[0], weight = edge[1];
            if (dist[u] + weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.offer(new int[]{dist[v], v});
            }
        }
    }
    return dist;
}
```

Interview Tips

Pattern Recognition

- When to identify this technique in problem statements
- Key indicators and problem characteristics
- Similar problem types and variations

Common Mistakes

- Implementation pitfalls to avoid
- Edge cases that are often missed
- Performance considerations

Implementation Variations

```
// Alternative approaches and optimizations
// Template code for common variations
```

Time & Space Complexity

Algorithm	Time Complexity	Space Complexity
BFS/DFS	O(V + E)	O(V)
Dijkstra	O((V + E) log V)	O(V)
Bellman-Ford	O(VE)	O(V)
Kruskal MST	O(E log E)	O(V)
Prim MST	O(E log V)	O(V)
Topological Sort	O(V + E)	O(V)

Where V = vertices, E = edges.

Common Problem Patterns

1. Number of Islands (BFS/DFS)

Problem: Count number of islands in a 2D grid.

```
public int numIslands(char[][] grid) {
    int count = 0;
    for (int i = 0; i < grid.length; i++) {
        for (int j = 0; j < grid[0].length; j++) {
            if (grid[i][j] == '1') {
                dfs(grid, i, j);
                count++;
            }
        }
    }
    return count;
```

```
    }

    private void dfs(char[][] grid, int i, int j) {
        if (i < 0 || i >= grid.length || j < 0 || j >= grid[0].length || grid[i][j] != '
        grid[i][j] = '0';
        dfs(grid, i+1, j); dfs(grid, i-1, j); dfs(grid, i, j+1); dfs(grid, i, j-1);
    }
```

2. Clone Graph

Problem: Deep clone a graph given a node reference.

```
    public Node cloneGraph(Node node) {
        if (node == null) return null;
        Map<Node, Node> map = new HashMap<>();
        return dfs(node, map);
    }

    private Node dfs(Node node, Map<Node, Node> map) {
        if (map.containsKey(node)) return map.get(node);
        Node clone = new Node(node.val);
        map.put(node, clone);
        for (Node neighbor : node.neighbors) {
            clone.neighbors.add(dfs(neighbor, map));
        }
        return clone;
    }
```

3. Course Schedule (Topological Sort)

Problem: Determine if you can finish all courses given prerequisites.

```
    public boolean canFinish(int numCourses, int[][] prerequisites) {
        int[] inDegree = new int[numCourses];
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());

        for (int[] pre : prerequisites) {
            graph.get(pre[1]).add(pre[0]);
            inDegree[pre[0]]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0) queue.offer(i);
        }

        int count = 0;
```

```
        while (!queue.isEmpty()) {
            int course = queue.poll();
            count++;
            for (int next : graph.get(course)) {
                if (--inDegree[next] == 0) queue.offer(next);
            }
        }
        return count == numCourses;
    }
```

4. Network Delay Time (Dijkstra)

Problem: Find time for signal to reach all nodes.

```
    public int networkDelayTime(int[][] times, int n, int k) {
        Map<Integer, List<int[]>> graph = new HashMap<>();
        for (int[] t : times) {
            graph.computeIfAbsent(t[0], x -> new ArrayList<>()).add(new int[]{t[1], t[2]
        }

        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]);
        pq.offer(new int[]{k, 0});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int u = curr[0], d = curr[1];
            if (d > dist[u]) continue;

            if (graph.containsKey(u)) {
                for (int[] edge : graph.get(u)) {
                    int v = edge[0], w = edge[1];
                    if (dist[u] + w < dist[v]) {
                        dist[v] = dist[u] + w;
                        pq.offer(new int[]{v, dist[v]});
                    }
                }
            }
        }

        int maxTime = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1;
            maxTime = Math.max(maxTime, dist[i]);
        }
        return maxTime;
    }

    ---
```

```
[← Previous: Dynamic Programming](09_Dynamic_Programming.md) | [Back to Main Guide](
```